

ABSTRACT

Introductory programming courses provide an essential foundation for advanced study in computer science, data science, artificial intelligence, cybersecurity, and scientific computing. This project presents a redesign of an introductory Python programming course through FacultyHack@Gateways 2026, with the goal of introducing students to high-performance computing (HPC), Science Gateway technologies, reproducible workflows, and data-intensive problem solving. The course serves students with foundational programming knowledge and covers Python concepts such as data types, control structures, functions, arrays, inheritance, object-oriented design, and introductory machine learning applications. The redesigned curriculum extends traditional programming instruction by introducing computational efficiency, scalability, HPC, and research cyberinfrastructure through hands-on labs, real-world datasets, and practical projects. Science Gateway resources are used to reduce technical barriers and provide students with early access to advanced computing environments. The expected impact includes improved confidence with computational tools, greater familiarity with scalable and reproducible workflows, and stronger preparation for upper-level coursework, undergraduate research, and computing careers. The project demonstrates that HPC and research cyberinfrastructure can be introduced effectively at the introductory level when activities are carefully scoped, accessible, and connected to familiar Python programming tasks.

INTRODUCTION

CSC 291 introduces students with foundational programming experience to Python and prepares them for advanced computing, data science, and software development. Traditionally, students completed relatively small programming exercises using local or cloud notebook environments. FacultyHack motivated a redesign that embeds HPC, research cyberinfrastructure, authentic datasets, and structured GenAI-assisted debugging directly into existing Python activities. The goal is to move students from isolated programming exercises toward scalable, reproducible, and data-intensive computational experiences while preserving core Python learning objectives..

Science Gateway Resources used

- Students use Jetstream2 to run a Python data-analysis workflow beyond local/Colab computing and compare execution.
- Cornell Virtual Workshop + HPC Carpentry support course development and scaffold introductory HPC concepts.
- Design choice: one primary computing environment plus one meaningful scalable-computing experience.

Mentor Feedback Incorporated

- Embed HPC within Python: integrate scalable-computing activities into existing programming tasks, not as a standalone topic.
- Structure GenAI use: students accept, modify, or reject debugging suggestions and explain why.
- Use authentic datasets: larger real-world data demonstrates when computing beyond the desktop is useful.
- Focus the environment: one primary platform reduces cognitive load for introductory students.

PROJECTED IMPACT ON STUDENTS

Broader Access: Provides early exposure to HPC, research cyberinfrastructure, and advanced computing resources.

Practical Skills: Develops hands-on experience with authentic datasets, machine learning applications, and reproducible workflows.

Computing Confidence: Expected to increase students' confidence in addressing larger and more computationally demanding problems.

Career & Research Readiness: Supports preparation for upper-level coursework, undergraduate research, and careers in AI, data science, cybersecurity, and scientific computing.

TARGET COURSE DESCRIPTION

Course Name	Introduction to Python Programing
Course Number	CSC 291
Department	Computer Science
Enrollment	25
Prerequisites	CSC152 (Java) pass with a grade of “C” or better.
Project Repo	https://github.com/Josephogo/joseph-aneke

Course Description

This course introduces students who already understand fundamental programming concepts to the Python programming language. Students will explore data types, control structures, functions, parameter passing, library functions, arrays, inheritance, and object-oriented design. They will solve problems, address real-world software development challenges, and develop practical applications, including introductory machine learning projects. In addition to programming fundamentals, students will gain early exposure to computational efficiency, scalability, high-performance computing (HPC), and research cyberinfrastructure. Through hands-on projects, laboratory activities, and enrichment exercises, students will work with real-world datasets and computational resources to develop reproducible and well-documented projects. Students will also explore how science gateways, supercomputing systems, and other advanced computing resources can support data-intensive applications and the solution of real-world problems.

LESSONS LEARNED

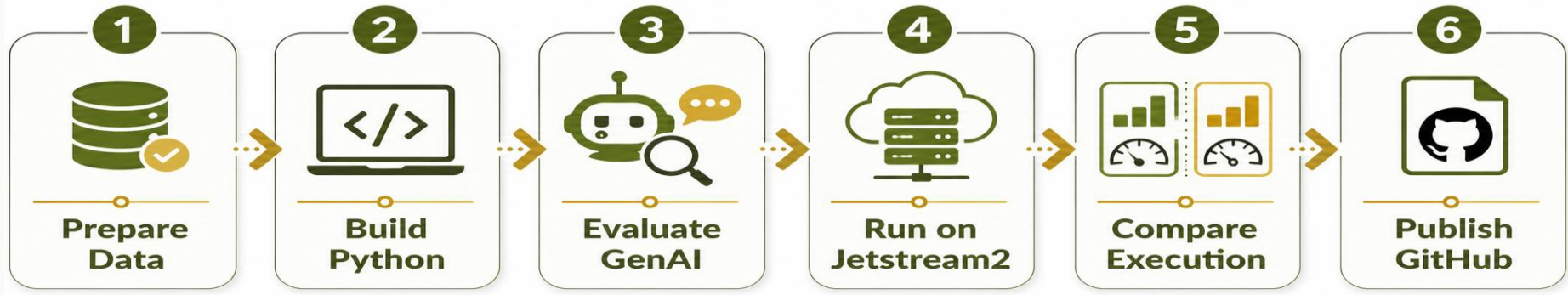
- Integrate, don't add: embed HPC in familiar Python tasks.
- Focus beats breadth: one platform plus one meaningful experience is enough.
- AI needs structure: students should evaluate and justify AI suggestions.
- Scale creates context: larger datasets make efficiency and scalability tangible.

REFERENCES

- Jetstream2 Documentation, ACCESS.
- Cornell Virtual Workshop, Cornell Center for Advanced Computing.
- HPC Carpentry, introductory HPC teaching materials.

FINAL PROJECT / DELIVERABLE

- 1. Prepare:** authentic real-world dataset.
- 2. Build:** Python analysis/visualization or introductory ML workflow.
- 3. Evaluate GenAI:** debug code; accept, modify, or reject suggestions with rationale.
- 4. Scale:** execute on Jetstream2 and compare with local/Colab.
- 5. Reproduce:** publish code, results, README, comparison, and AI-use reflection in GitHub



Project Goals

- Execute and compare:** Python programs in local and Jetstream2 computing environments.
- Process and analyze** an authentic large dataset using shared cyberinfrastructure.
- Critically evaluate** GenAI-generated debugging suggestions for correctness and efficiency.
- Compare execution behavior** to determine when additional computing resources are beneficial.
- Document and publish** a reproducible Python workflow through GitHub.

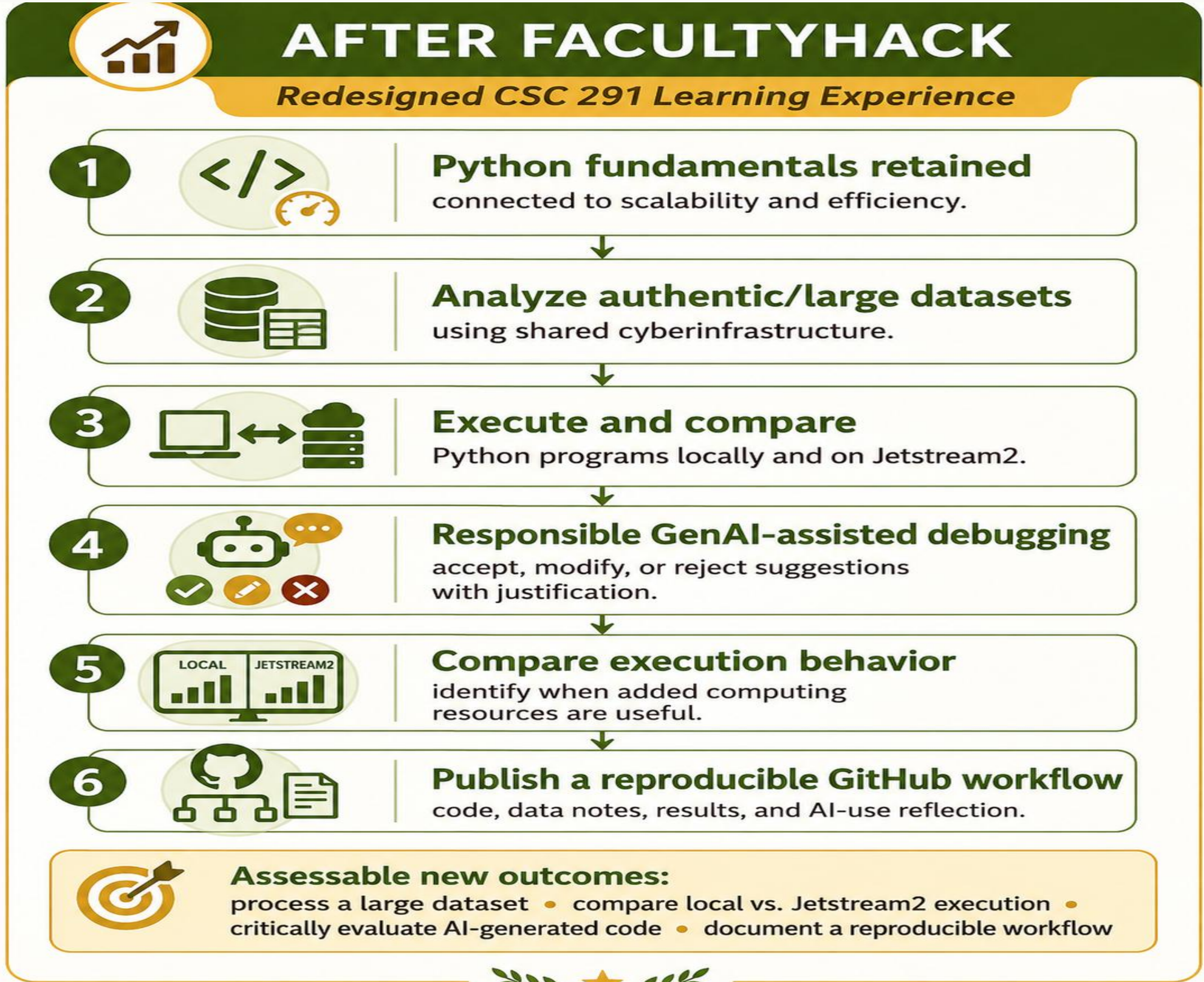
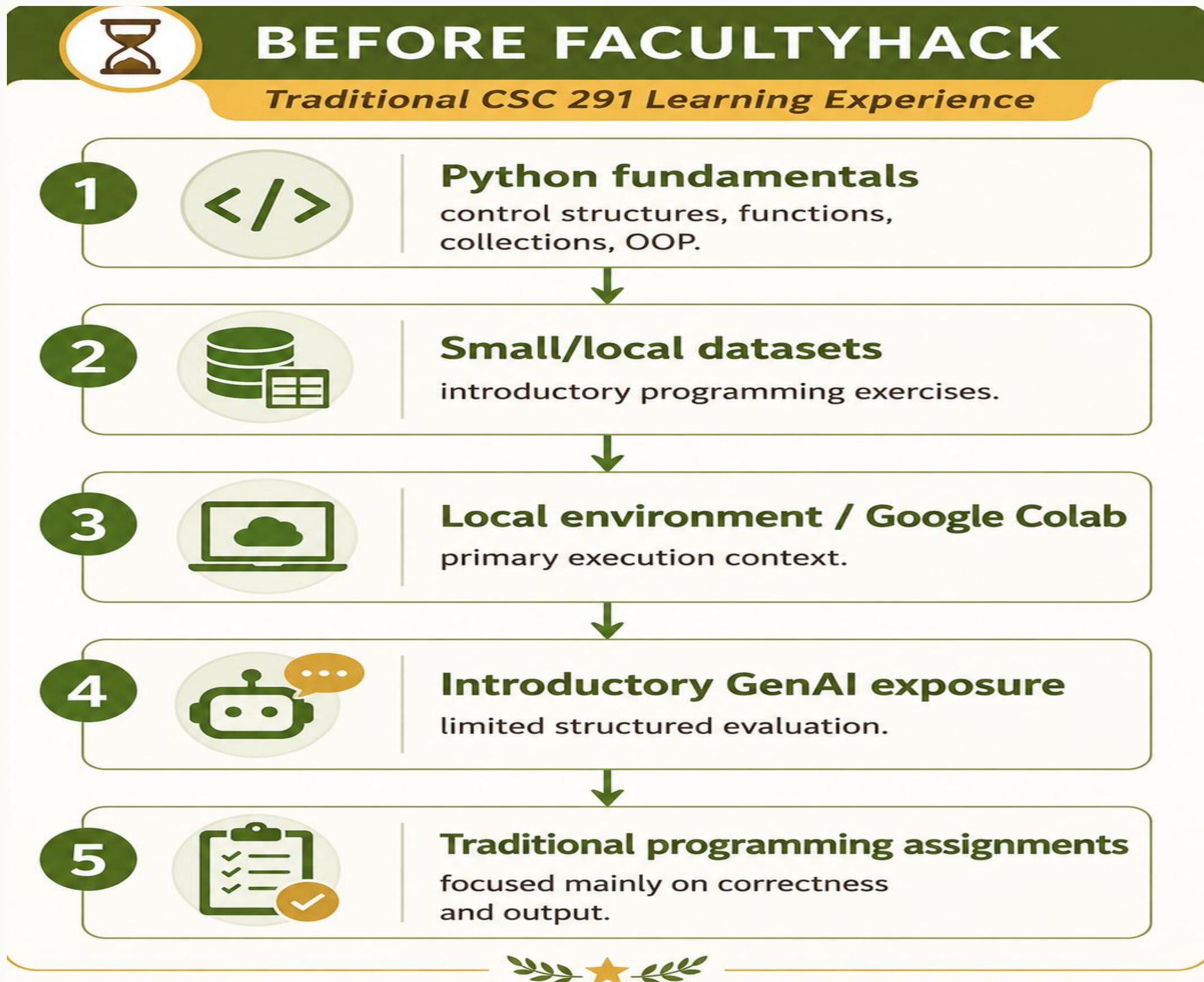
EXPANSION IDEAS

- Extend scalable-computing activities into Data Structures, cybersecurity, HCI, and AI courses.
- Introduce larger datasets and parallel-processing activities as students progress.
- Build cross-course projects and guest activities with research-computing partners.

Grading Policy

- | | |
|-----------------------------------|-----|
| Attendance | 5% |
| Discussion- class-participation | 5% |
| Ethics Paper | 5% |
| Assignment | 15% |
| Project | 35% |
| Exams (tests, quizzes, and final) | 35% |

COURSE REDESIGN: BEFORE & AFTER



AUTHORS & MENTORS



Dr. Joseph Aneke
Faculty Participant
Department of Computer Science
Hampton University
Joseph.aneke@hamptonu.edu



Dr. Sabrina Perry
Faculty Mentor
Computer Science and Information Technology
Austin Peay State University
perrys1@apsu.edu



PROJECT GITHUB REPO

github.com/Josephogo/joseph-aneke

Sponsors: SGX3 · National Science Foundation · ORNL · Omnibond · TACC · HackHPC



**FacultyHack
@Gateways26**

hackhpc.github.io/facultyhack-gateways26

